



Roofline

2025

André Pereira

Goals of the Roofline Model



CONVENTIONAL WISDOM IN computer architecture produced similar designs. Nearly every desktop and server computer uses caches, pipelining, superscalar instruction issue, and out-of-order execution. Although the instruction sets varied, the microprocessors were all from the same school of

Roofline Model

For the foreseeable future, off-chip memory bandwidth will often be the constraining resource in system performance.²³ Hence, we want a model that relates processor performance to off-chip memory traffic. Toward this

DOI:10.1145/1498765.1498785

The Roofline model offers insight on how to improve the performance of software and hardware.

BY SAMUEL WILLIAMS, ANDREW WATERMAN, AND DAVID PATTERSON

Roofline:



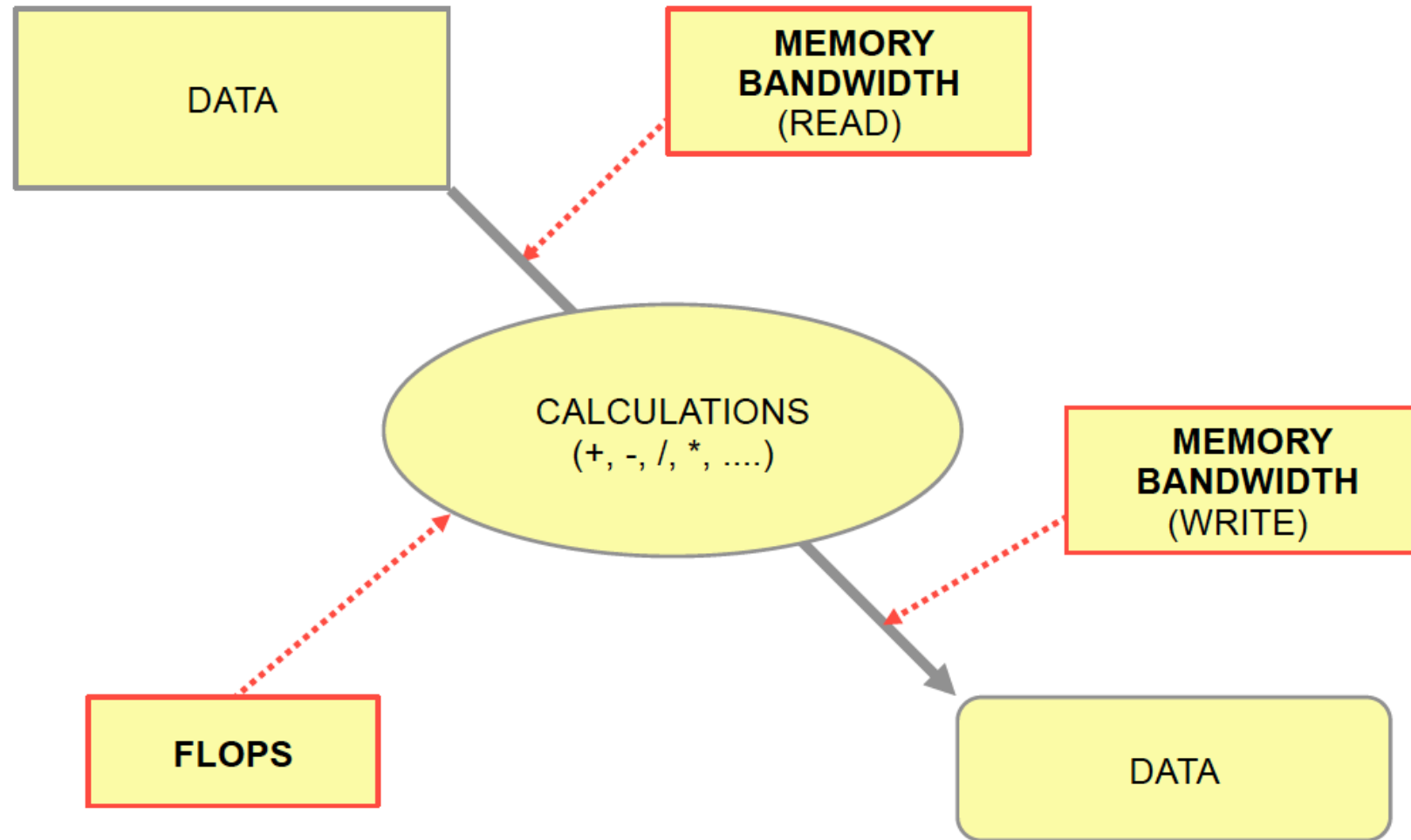
Performance Limiting Factors



Leopold Grinberg

IBM, T.J. Watson Research Center, USA

ICSC 2014, Shanghai, China





- ❖ There are three principal components to performance:
 - **Computation**
 - **Communication**
 - **Locality**
- ❖ Each architecture has a different balance between these
- ❖ Each kernel has a different balance between these
- ❖ Performance is a question of how well ~~an~~ kernel's characteristics map to an architecture's characteristics

The Roofline Model:

A pedagogical tool for program analysis and optimization



- ❖ For us, floating point performance (**Gflop/s**) is the metric of interest (typically double precision) ... but we could also consider SP or int
- ❖ Peak in-core performance can only be attained if:
 - fully exploit ILP, DLP, FMA, etc...
 - non-FP instructions don't sap instruction bandwidth
 - threads don't diverge (GPUs)
 - transcendental/non pipelined instructions are used sparingly
 - branch mispredictions are rare
- ❖ To exploit a form of in-core parallelism, it must be:
 - Inherent in the algorithm
 - Expressed in the high level implementation
 - Explicit in the generated code

The Roofline Model:

A pedagogical tool for program analysis and optimization



- ❖ For us, DRAM bandwidth (**GB/s**) is the metric of interest
and cache!!!
- ❖ Peak bandwidth can only be attained if certain optimizations are employed:
 - Few unit stride streams
 - NUMA allocation and usage
 - SW Prefetching
 - Memory Coalescing (GPU)

The Roofline Model:

A pedagogical tool for program analysis and optimization



- ❖ Computation is free, Communication is expensive.
- ❖ Maximize locality to minimize communication
- ❖ **There is a lower limit to communication: compulsory traffic**

- ❖ Hardware changes can help minimize communication
 - Larger cache capacities minimize capacity misses
 - Higher cache associativities minimize conflict misses
 - Non-allocating caches minimize compulsory traffic

- ❖ Software optimization can also help minimize communication
 - Padding avoids conflict misses
 - Blocking avoids capacity misses
 - Non-allocating stores minimize compulsory traffic

3Cs model
for caches

The Roofline Model:

A pedagogical tool for program analysis and
optimization



❖ Temporal Locality

- reusing data (either registers or cache lines) multiple times
- amortizes the impact of limited bandwidth.
- **transform loops or algorithms to maximize reuse.**

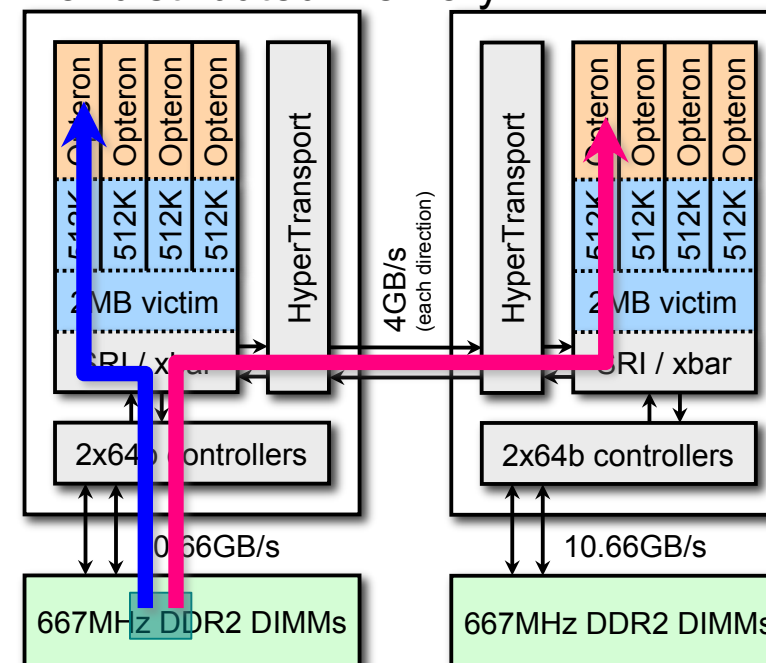
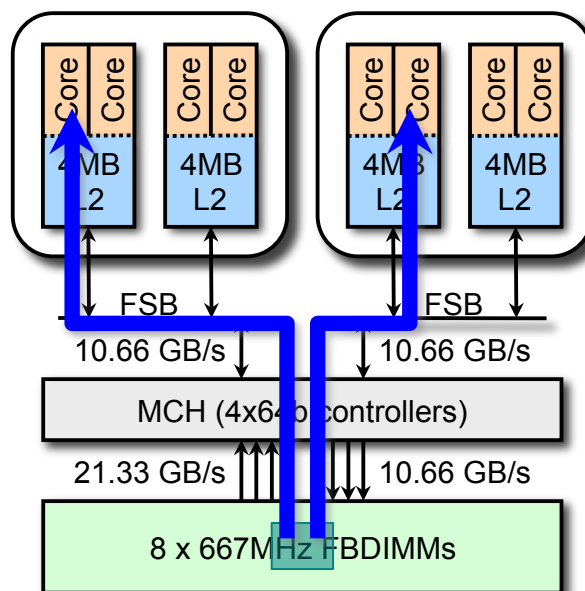
❖ Spatial Locality

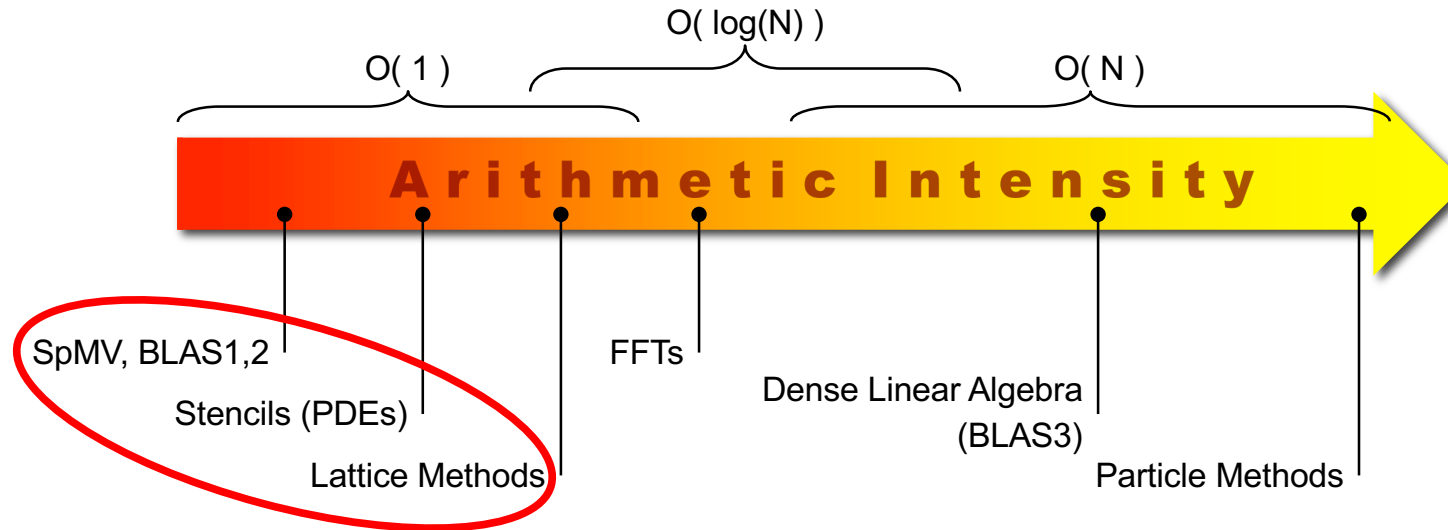
- data is transferred from cache to registers in words.
- However, data is transferred to the cache in 64-128Byte lines
- using every word in a line maximizes spatial locality.
- **transform data structures into *structure of arrays (SoA)* layout**

❖ Sequential Locality

- Many memory address patterns access cache lines sequentially.
- CPU's hardware stream prefetchers exploit this observation to hide speculatively load data to memory latency.
- **Transform loops to generate (a few) long, unit-stride accesses.**

- ❖ Recent multicore SMPs have integrated the memory controllers on chip.
- ❖ As a result, memory-access is non-uniform (NUMA)
- ❖ That is, the bandwidth to read a given address varies dramatically among between cores
- ❖ Exploit NUMA (affinity+first touch) when you malloc/init data.
- ❖ Concept is similar to data decomposition for distributed memory

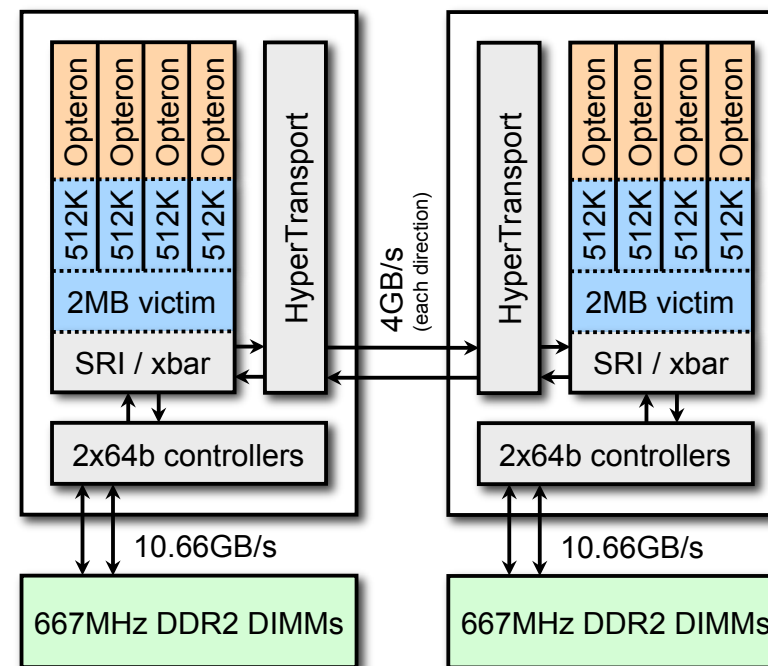




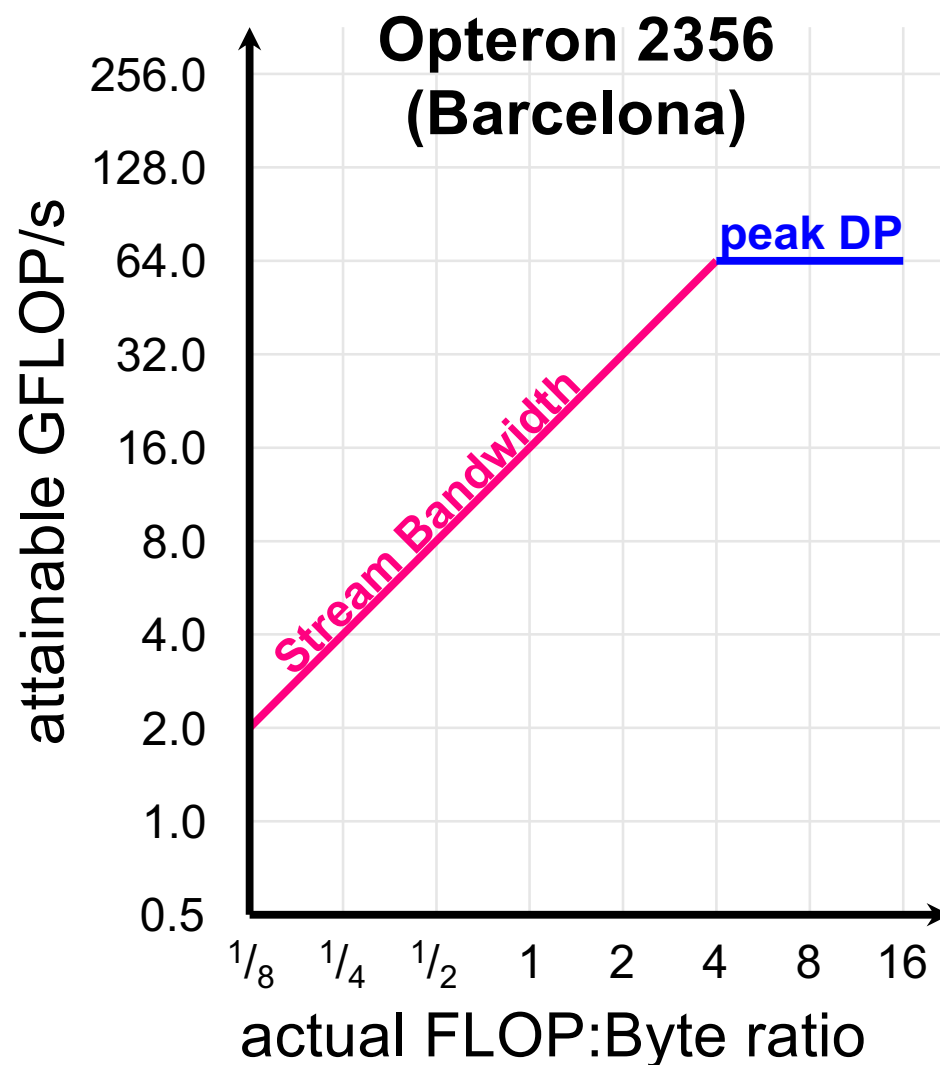
- ❖ **True Arithmetic Intensity (AI) ~ Total Flops / Total DRAM Bytes**
- ❖ Some HPC kernels have an arithmetic intensity that scales with problem size (increased temporal locality)
- ❖ Others have constant intensity
- ❖ Arithmetic intensity is ultimately limited by compulsory traffic
- ❖ Arithmetic intensity is diminished by conflict or capacity misses.



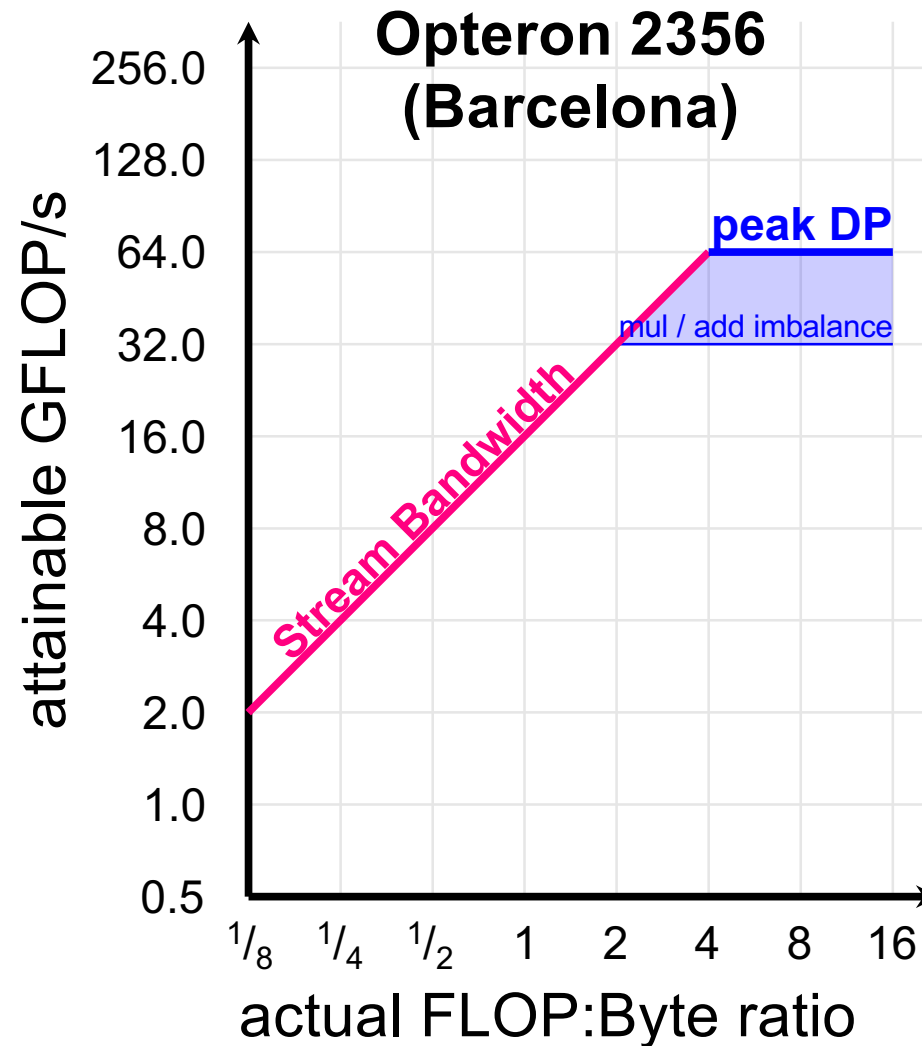
- ❖ Consider the Opteron 2356:
 - Dual Socket (NUMA)
 - limited HW stream prefetchers
 - quad-core (8 total)
 - 2.3GHz
 - 2-way SIMD (DP)
 - separate FPMUL and FPADD datapaths
 - 4-cycle FP latency



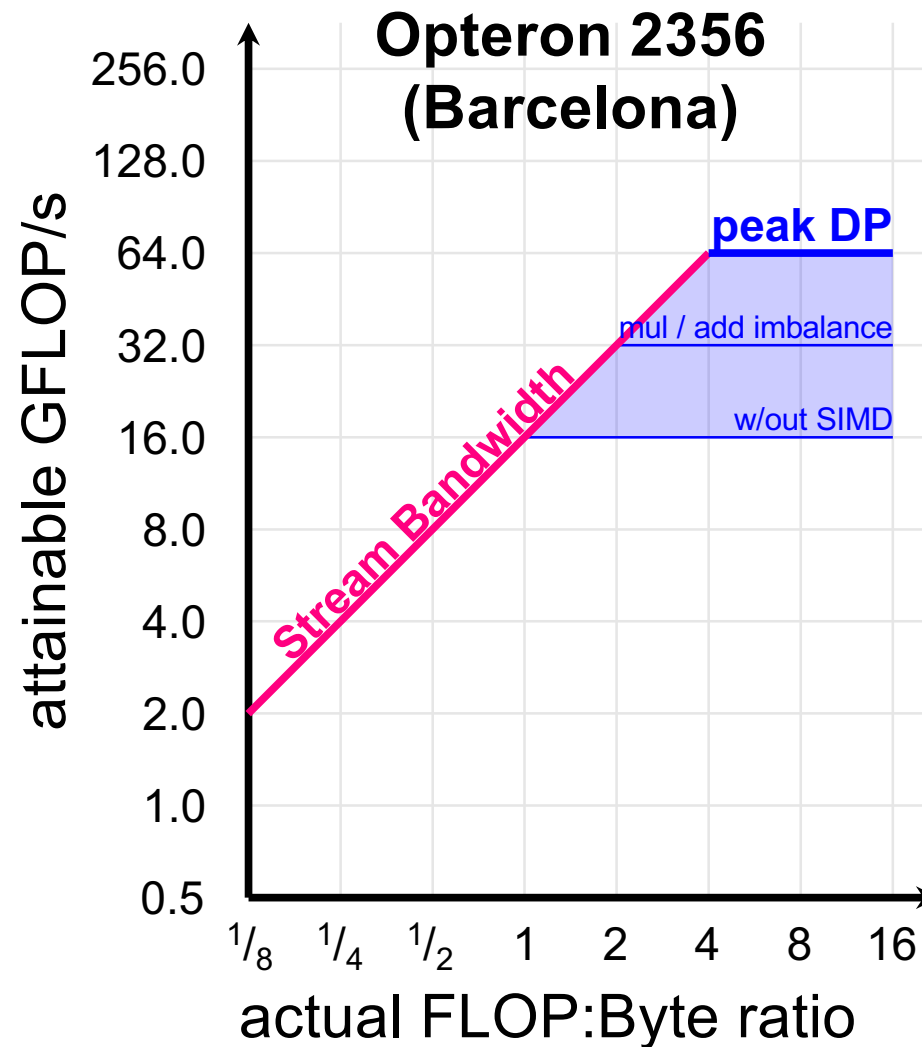
- ❖ Assuming **expression of parallelism** is the challenge on this architecture, what would the roofline model look like ?



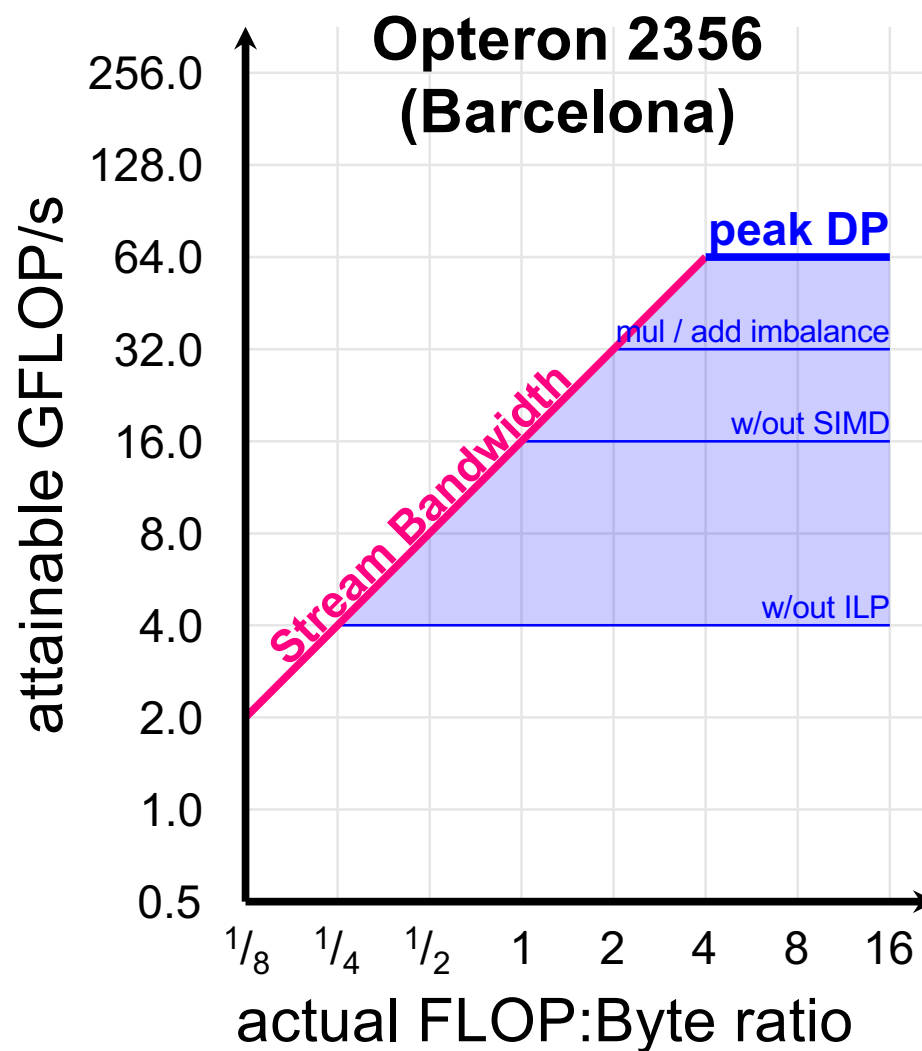
- ❖ Plot on log-log scale
- ❖ Given AI, we can easily bound performance
- ❖ But architectures are much more complicated
- ❖ We will bound performance as we eliminate specific forms of in-core parallelism



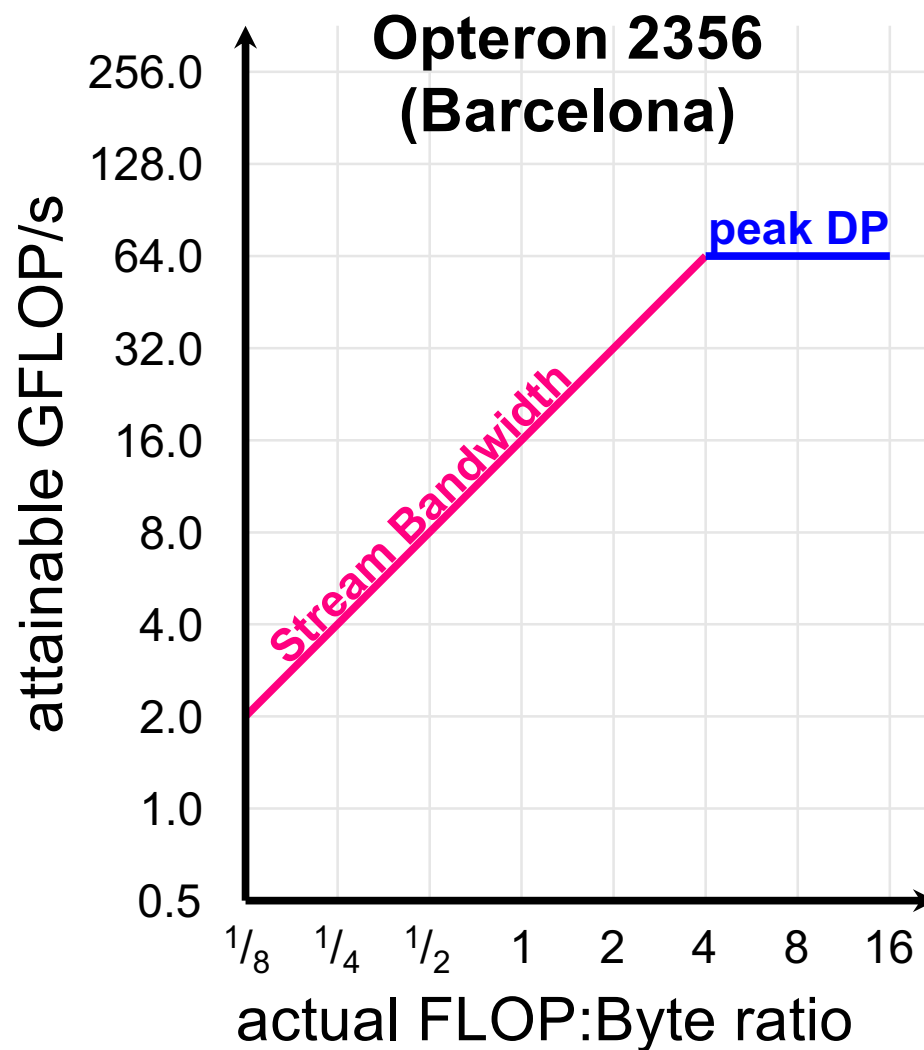
- ❖ Opterons have dedicated multipliers and adders.
- ❖ If the code is dominated by adds, then attainable performance is half of peak.
- ❖ We call these **Ceilings**
- ❖ They act like constraints on performance



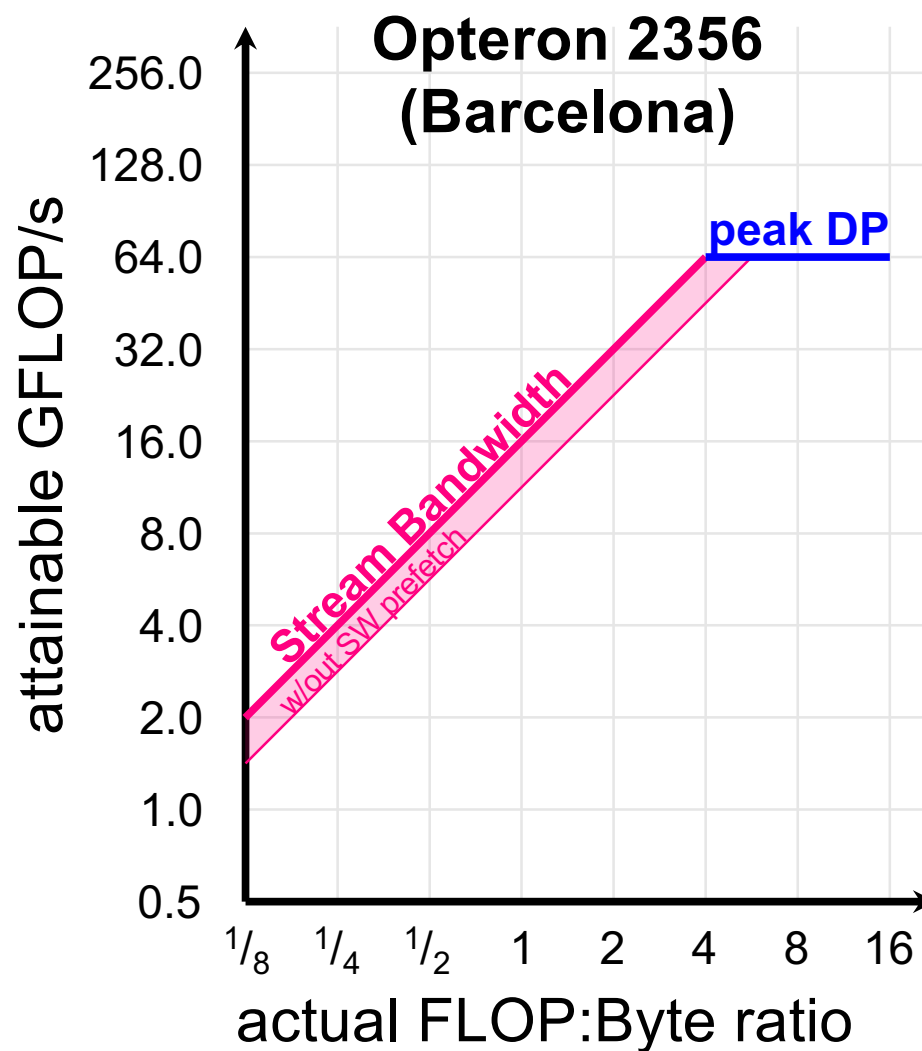
- ❖ Opterons have 128-bit datapaths.
- ❖ If instructions aren't SIMDized, attainable performance will be halved



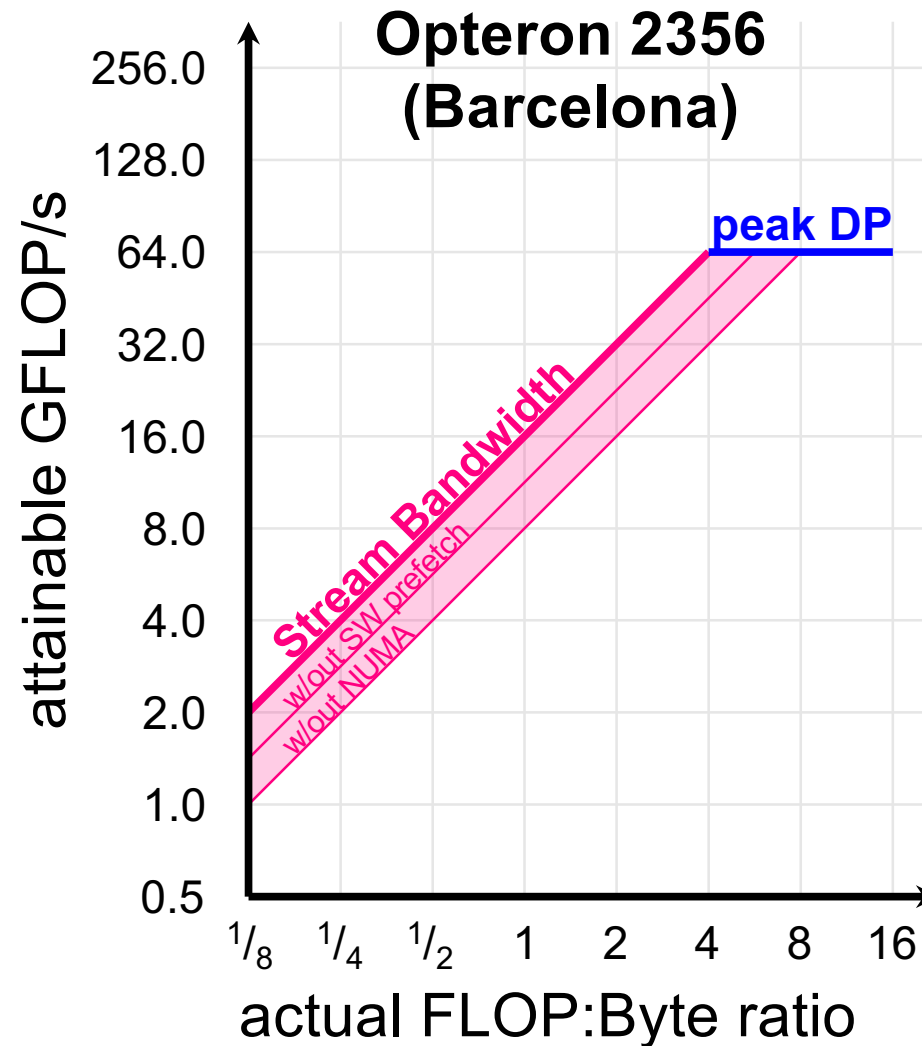
- ❖ On Opterons, floating-point instructions have a 4 cycle latency.
- ❖ If we don't express 4-way ILP, performance will drop by as much as 4x



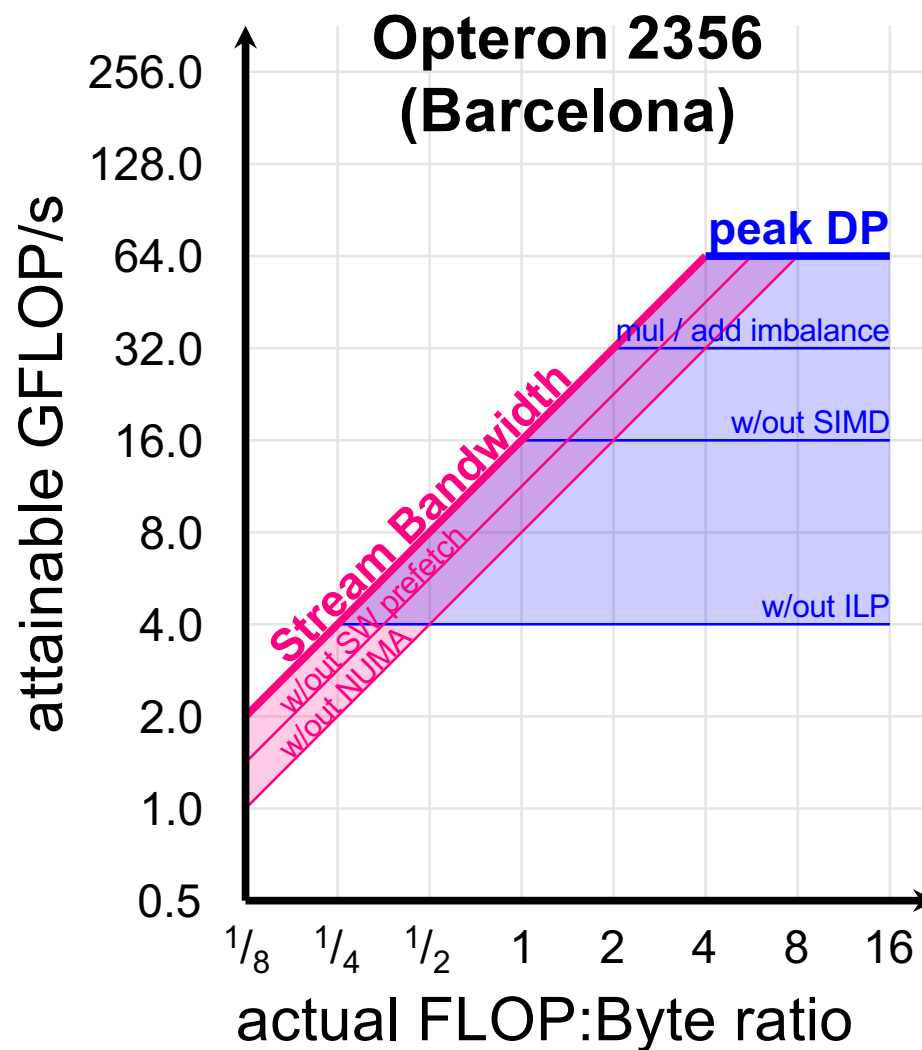
- ❖ We can perform a similar exercise taking away parallelism from the memory subsystem



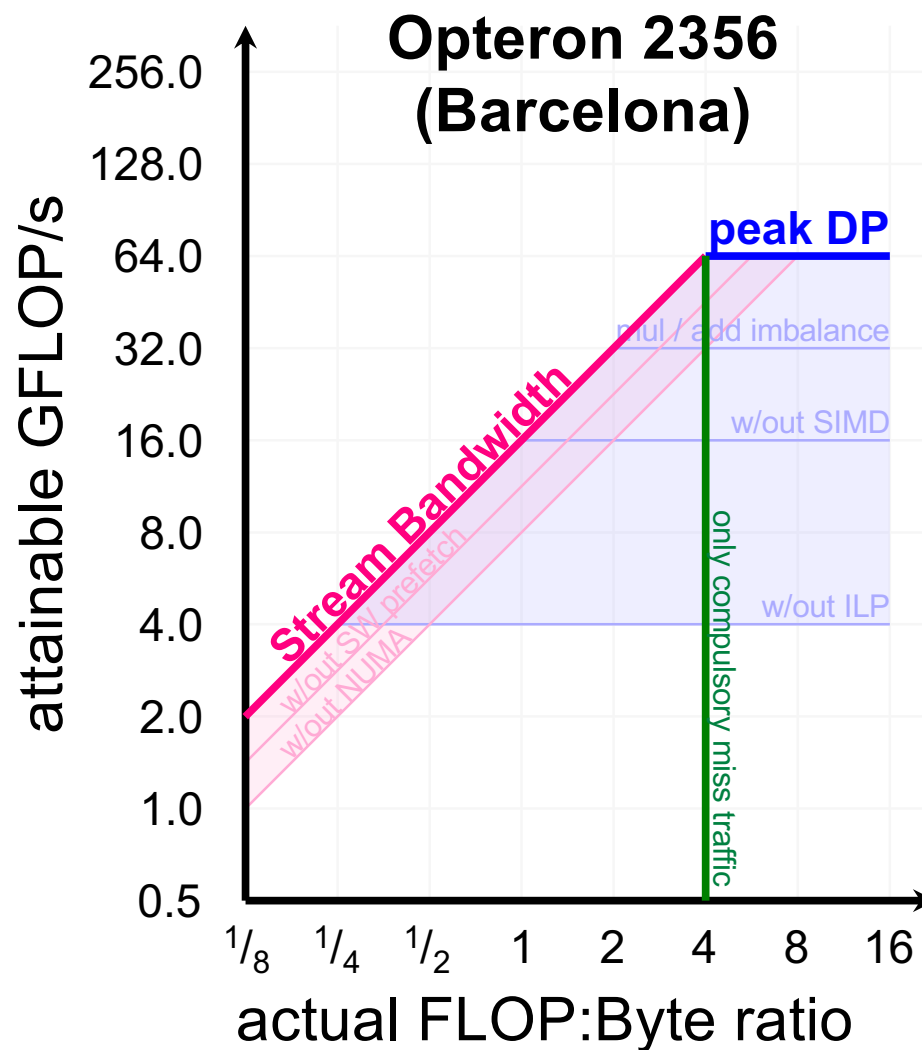
- ❖ Explicit software prefetch instructions are required to achieve peak bandwidth



- ❖ Opterons are NUMA
- ❖ As such memory traffic must be correctly balanced among the two sockets to achieve good Stream bandwidth.
- ❖ We could continue this by examining strided or random memory access patterns

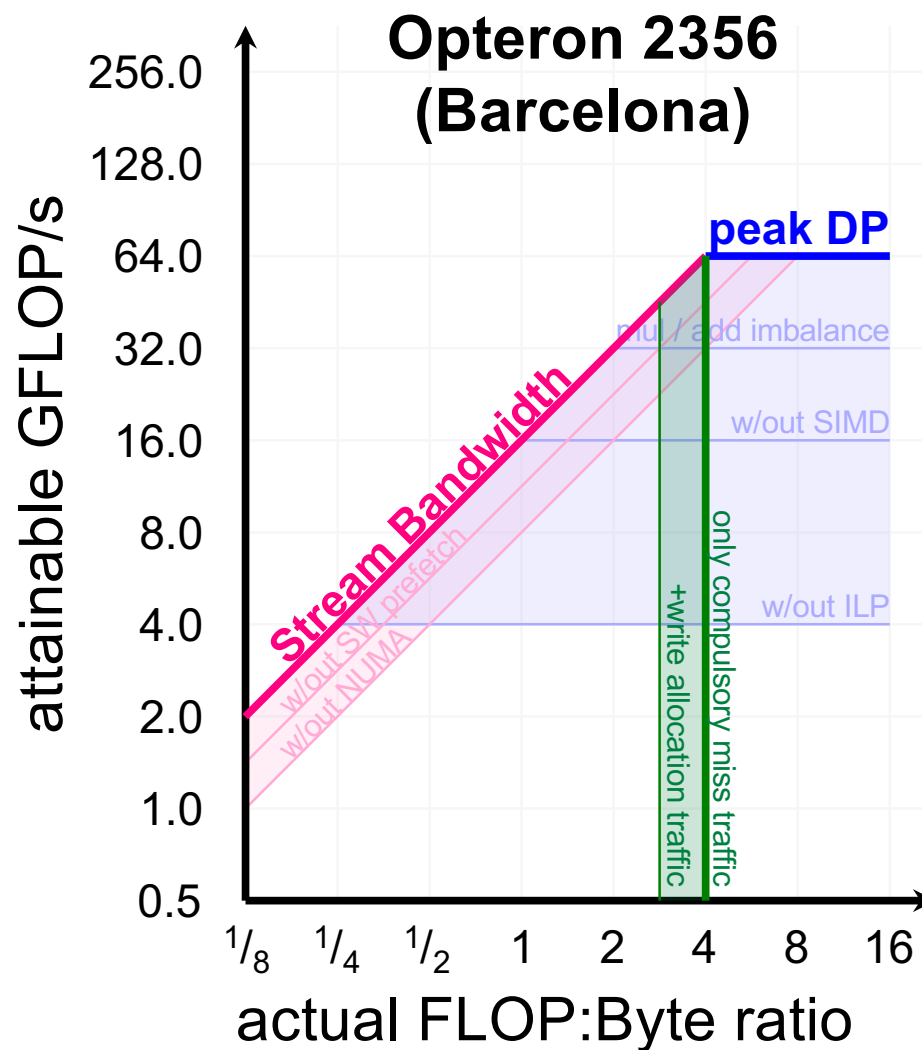


- ❖ We may bound performance based on the combination of expressed in-core parallelism and attained bandwidth.



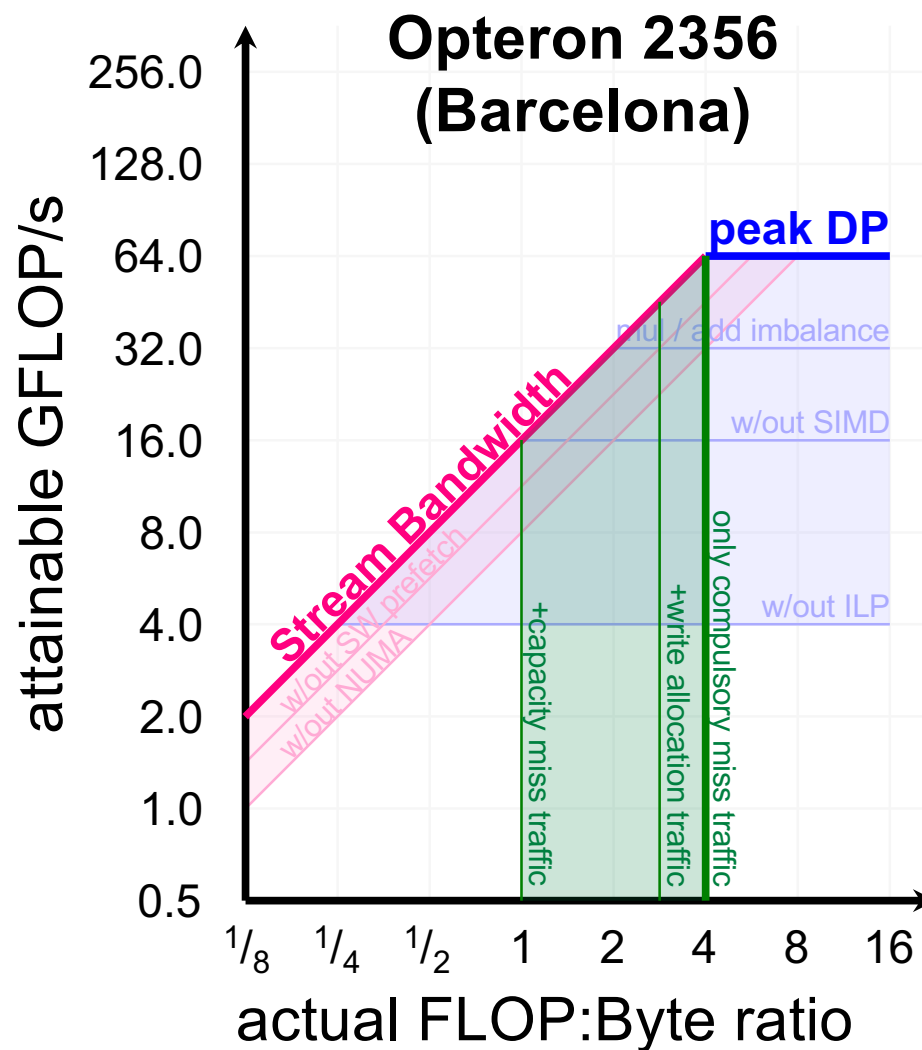
- ❖ Remember, memory traffic includes more than just compulsory misses.
- ❖ As such, actual arithmetic intensity may be substantially lower.
- ❖ Walls are unique to the architecture-kernel combination

$$AI = \frac{\text{FLOPs}}{\text{Compulsory Misses}}$$



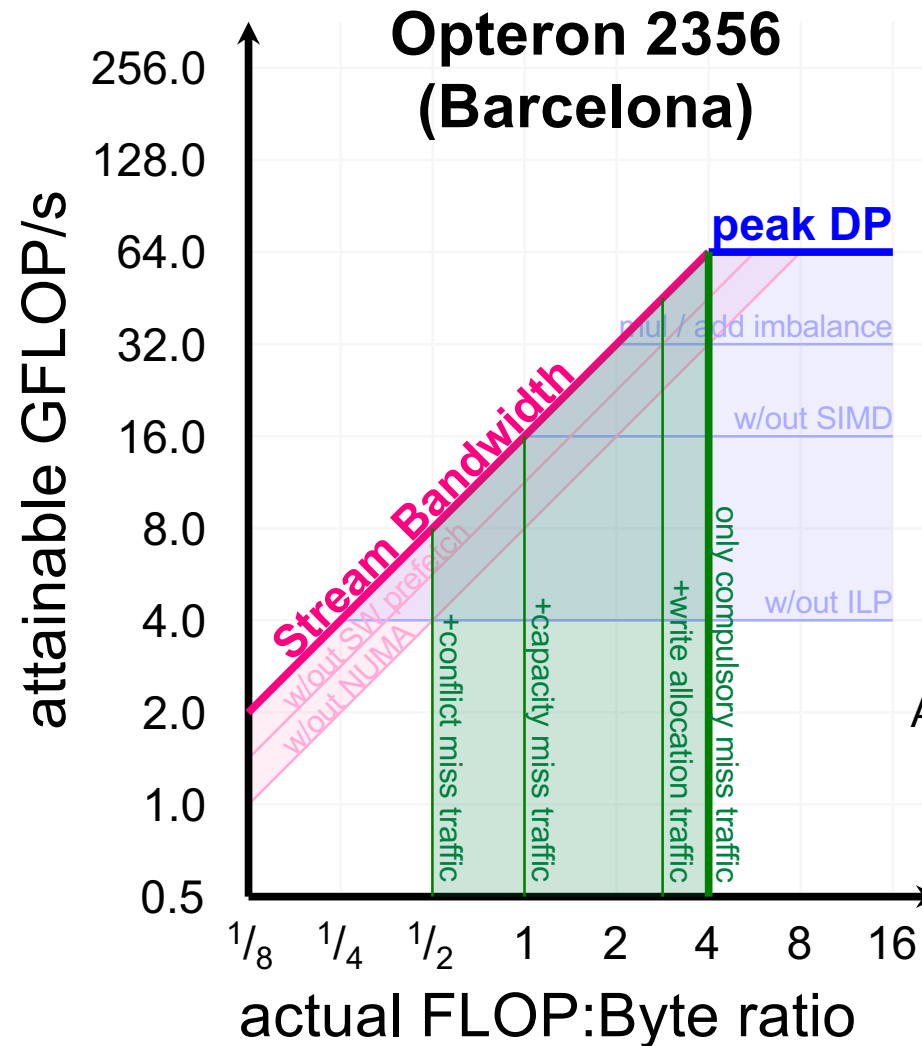
- ❖ Remember, memory traffic includes more than just compulsory misses.
- ❖ As such, actual arithmetic intensity may be substantially lower.
- ❖ Walls are unique to the architecture-kernel combination

$$AI = \frac{\text{FLOPs}}{\text{Allocations} + \text{Compulsory Misses}}$$



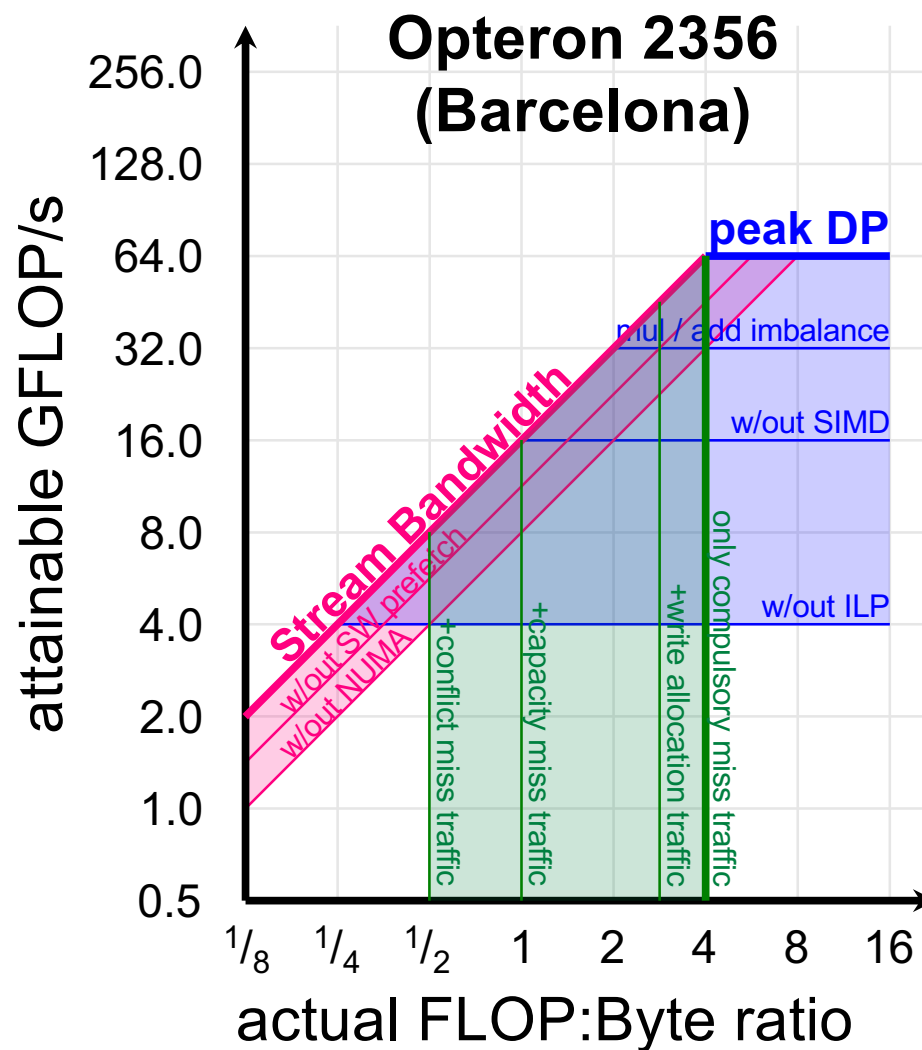
- ❖ Remember, memory traffic includes more than just compulsory misses.
- ❖ As such, actual arithmetic intensity may be substantially lower.
- ❖ Walls are unique to the architecture-kernel combination

$$AI = \frac{\text{FLOPs}}{\text{Capacity} + \text{Allocations} + \text{Compulsory}}$$



- ❖ Remember, memory traffic includes more than just compulsory misses.
- ❖ As such, actual arithmetic intensity may be substantially lower.
- ❖ Walls are unique to the architecture-kernel combination

$$AI = \frac{\text{FLOPs}}{\text{Conflict} + \text{Capacity} + \text{Allocations} + \text{Compulsory}}$$

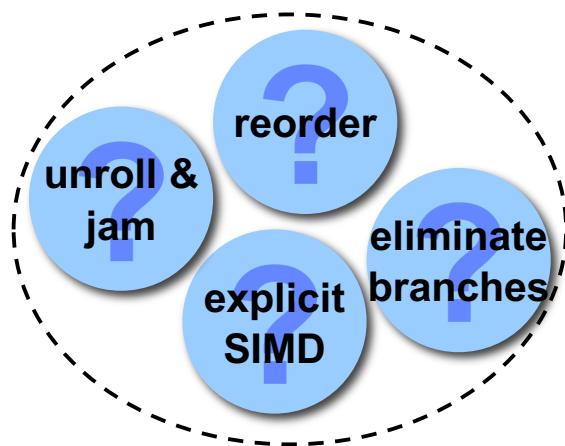


- ❖ Optimizations remove these walls and ceilings which act to constrain performance.



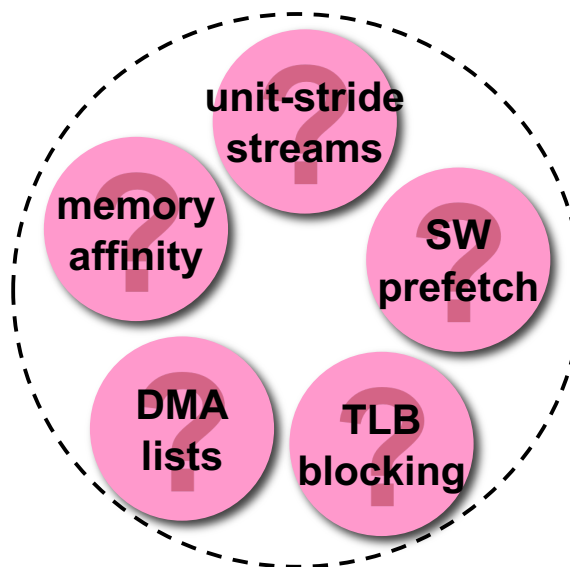
Maximizing In-core Performance

- Exploit in-core parallelism (ILP, DLP, etc...)
- Good (enough) floating-point balance



Maximizing Memory Bandwidth

- Exploit NUMA
- Hide memory latency
- Satisfy Little's Law



Minimizing Memory Traffic

- Eliminate:
- Capacity misses
 - Conflict misses
 - Compulsory misses
 - Write allocate behavior

